checkpoint.py — technical

`src/moe_pruning/serialization/checkpoint.py · 406 lines · moe_pruning.serialization.checkpoint`

Purpose

Full and delta checkpoint formats with strict validation.

Step by step

Classes and functions in source order. Signatures, line ranges, raised exceptions and settings are extracted from the code; numbered steps describe what the code does.

`class CheckpointError`

lines 62-63 · bases RuntimeError

Raised for invalid, inconsistent or unsupported checkpoints.

`deduplicate_state_dict`

```
def deduplicate_state_dict(state: dict[str, torch.Tensor]) -> tuple[dict[str, torch.Tensor], dict[str, str]]
```

lines 70-83

Keep one copy of tensors sharing storage; return (tensors, {alias: canonical}).

1. Process keys in order of fewest dots then name; key tensors by (device, storage pointer, offset, shape, dtype).
2. First occurrence is stored (contiguous, CPU); later ones become `alias -> canonical` entries (tied weights).

`layer_method`

```
def layer_method(entry: dict) -> str
```

lines 86-88

Replacement method of a manifest layer entry (older manifests: bias).

`replaced_expert_prefixes`

```
def replaced_expert_prefixes(manifest: dict) -> list[str]
```

lines 91-96

State-dict prefixes of all removed experts in a manifest.

`expected_bias_keys`

```
def expected_bias_keys(manifest: dict) -> list[str]
```

lines 99-106

State-dict keys that must exist: the biases of bias-replaced experts.

`parameters_on_meta`

```
def parameters_on_meta()
```

lines 110-129

Create parameters on the meta device but buffers normally.

1. Temporarily patch `nn.Module.register_parameter` so every new parameter is moved to `meta` (no memory, no init cost) while buffers stay real (e.g. rotary `inv_freq`, not stored in checkpoints).

`removed_weight_violations`

```
def removed_weight_violations(adapter: MoEAdapter, model: nn.Module, state_shapes: dict[str, tuple[int, ...]], manifest: dict) -> list[str]
```

lines 132-140

Stored tensors (by name/shape) that hold weights of experts the manifest lists as removed.

`stored_tensor_shapes`

```
def stored_tensor_shapes(path: Path) -> dict[str, tuple[int, ...]]
```

lines 143-148

Name -> shape of every tensor in a safetensors file, without loading data.

`save_pruned_model`

```
def save_pruned_model(model: nn.Module, tokenizer, output_dir: Path, manifest: dict, experiment_config:
dict, readme: str | None=None, checkpoint_format: str='full') -> Path
```

lines 151-192 · raises CheckpointError, ValueError

Write the checkpoint and verify that no replaced expert weights are stored.

1. Delta format -> `_save_delta`.
2. Full: deduplicate the state dict; `removed_weight_violations` on stored names/shapes must be empty; every expected bias key must exist.
3. Write `model.safetensors`, then metadata (`config.json`, generation config, tokenizer, manifest with format, aliases and stored sizes, experiment config, README).

`_save_delta`

```
def _save_delta(model, tokenizer, output_dir: Path, manifest: dict, experiment_config: dict, readme) ->
Path
```

lines 205-233 · raises CheckpointError

1. Take only `expected_bias_keys` from the state dict; record `base_model` (source, revision, dtype) in the manifest; write `replacements.safetensors` and the same metadata files.

`load_pruned_model`

```
def load_pruned_model(model_path: Path | str, device: torch.device | str='cpu', placement:
ModelPlacement | None=None, base_model_source: str | None=None, load_kwargs: dict | None=None)
```

lines 265-357 · raises CheckpointError

Rebuild a pruned model from a checkpoint written by `save_pruned_model`.

1. Read and check the manifest format/version; delta -> `_load_delta`.
2. Full: config -> plugin; build the model under `parameters_on_meta`; per manifest layer check state prefixes and install meta replacement modules via `replace_experts_in_layer`.
3. Load tensors, restore tied aliases; expected keys (from the rebuilt model) must equal stored keys exactly; shapes must match; `load_state_dict(strict=True, assign=True)`; `tie_weights()`; no meta tensors may remain.
4. Place the model (single device or dispatch), eval mode, adapter `validate_loaded`, reinstall reroute masks, load tokenizer.

`_load_delta`

```
def _load_delta(model_path: Path, manifest: dict, placement: ModelPlacement, base_source, load_kwargs:
dict)
```

lines 360-406 · raises CheckpointError

Base model + re-applied replacement + stored biases.

1. Load the base model (manifest source or override) with its recorded dtype, revision, load kwargs and placement; model type must match the saved config.
2. Stored tensor names must equal exactly the expected bias keys.
3. For each manifest layer: check prefixes, then `replace_experts` with that layer's method and stored biases (which also re-applies reroute masks and frees the removed experts).
4. `validate_loaded`, load tokenizer from the checkpoint directory.

Module constants

Name	Value
FORMAT_NAME	'moe-pruning/bias-replaced-experts'
FORMAT_VERSION	1
WEIGHTS_NAME	'model.safetensors'
DELTA_WEIGHTS_NAME	'replacements.safetensors'
CHECKPOINT_FORMATS	('full', 'delta')
MANIFEST_NAME	'pruning_manifest.json'
EXPERIMENT_CONFIG_NAME	'experiment_config.json'

Errors raised

Function	Exception	Message (f-string fields in braces)
save_pruned_model	ValueError	checkpoint_format must be one of {CHECKPOINT_FORMATS}
save_pruned_model	CheckpointError	Refusing to save original weights of replaced experts: {violations}
save_pruned_model	CheckpointError	Bias parameters missing from the model: {missing_biases}
_save_delta	CheckpointError	Bias parameters missing from the model: {missing}
load_pruned_model	CheckpointError	{model_path} has no {MANIFEST_NAME}; not a pruned checkpoint
load_pruned_model	CheckpointError	Unsupported checkpoint format {manifest.get('format')} v{manifest.get('format_version')}
load_pruned_model	CheckpointError	State dict mismatch. missing ({len(missing)}): {missing[:10]} unexpected ({len(unexpected)}): {unexpected[:10]}
load_pruned_model	CheckpointError	Tensor shape mismatch: {wrong[:10]}
load_pruned_model	CheckpointError	Tensors were not materialised: {leftover_meta[:10]}
load_pruned_model	CheckpointError	Manifest layer {layer_id} not found in the rebuilt architecture
load_pruned_model	CheckpointError	Tied alias {alias} refers to missing tensor {canonical}
load_pruned_model	CheckpointError	State prefix mismatch for {layer_id} expert {expert_id}
_load_delta	CheckpointError	Base model {source} is {model.config.model_type}, checkpoint is {config.model_type}
_load_delta	CheckpointError	Delta tensors mismatch: missing {sorted(expected - set(tensors))[:10]}, unexpected {sorted(set(tensors) - expected)[:10]}
_load_delta	CheckpointError	Manifest layer {layer_id} not found in the base model
_load_delta	CheckpointError	State prefix mismatch for {layer_id} expert {expert_id}